

Trabajo 2ara la casa

En parejas, convierte la GeoCam persistente en un módulo CRUD completo con relaciones, una segunda migración y archivos permanentes. Entrega antes de la clase de la Semana 8.

Tarcas

- ☐ **C1. Tabla de álbumes.** Crea `albums` (`id`, `name` único, `createdAt`) y agrega a `photos` la columna `albumId` como llave foránea opcional con `onDelete: 'set null'`. Al borrar un álbum, sus fotos quedan sin álbum, no se borran.
- ☐ **C2. Segunda migración.** Agrega a `photos` las columnas `note` (texto opcional) y `favorite` (`integer('favorite', { mode: 'boolean' })` con valor por defecto `false`). Genera la migración y pruébala sobre un celular que ya tenga fotos de la clase: los datos anteriores deben conservarse.
- ☐ **C3. CRUD completo.** Pantalla `app/foto/[id].tsx` para ver una foto, editar su nota, marcarla como favorita, moverla de álbum y eliminarla con confirmación.
- ☐ **C4. Búsqueda y filtros.** En la lista de fotos, un buscador por nota (`like`) y filtros por álbum y por favoritas, usando `useLiveQuery(consulta, [busqueda, albumId, soloFavoritas])` para que se recalcule al cambiar los filtros.
- ☐ **C5. Archivos permanentes.** Al guardar una foto, cópiala de la caché a la carpeta de documentos de la app y guarda en SQLite la nueva ruta. Al eliminar el registro, elimina también el archivo.
- ☐ **C6. Documentación.** `README.md` Con el diagrama de tablas (imagen o texto) y un GIF del flujo, más el `AI-LOG.md` de la semana.

™cfircrcncia 2ara C5

La API de `expo-file-system` cambió en los SDK recientes: la nueva se basa en las clases `File`, `Directory` y `Paths`, y la anterior quedó en `expo-file-system/legacy`. Verifica en la documentación de tu versión de SDK antes de copiar este esquema:

```
// services/photoFiles.ts
import { Directory, File, Paths } from 'expo-file-system';

const photosDir = new Directory(Paths.document, 'photos');

export function persistPhoto(cacheUri: string): string {
  if (!photosDir.exists) photosDir.create();
  const source = new File(cacheUri);
  const destination = new File(photosDir, `${Date.now()}.jpg`);
  source.copy(destination);
  return destination.uri;
}

export function deletePhotoFile(uri: string) {
  const file = new File(uri);
  if (file.exists) file.delete();
}
```

Esta es una fuente frecuente de alucinaciones: la IA suele mezclar

`FileSystem.copyAsync` (API anterior) con las clases nuevas. Si ocurre, repórtalo en el AI-LOG.

Tercera migración

Escribe una tercera migración que cree un índice sobre `photos.created_at` y mide con `console.time` la diferencia al listar 1.000 fotos de prueba insertadas con un script de seed. Es un adelanto de la Semana 11 (rendimiento).

Entrega semanal

Entrega el enlace a tu repositorio de GitHub con un módulo CRUD funcional sobre SQLite y Drizzle: el taller en clase más el trabajo para la casa, con la carpeta `drizzle/` versionada.

Antes de entregar, verifica

- ☐ Hay al menos dos migraciones generadas con `drizzle-kit generate` y ninguna editada a mano.
- ☐ Las fotos de antes de la segunda migración siguen ahí después de aplicarla.
- ☐ Las pantallas no importan nada de Drizzle: todo pasa por hooks y repositorios.
- ☐ Crear, leer, actualizar y borrar funcionan, y los filtros se actualizan solos.
- ☐ Las fotos se guardan en la carpeta de documentos y se borran junto con su registro.
- ☐ `README.md` y `AI-LOG.md` están en la raíz.

Errores de IA que debes detectar

La IA genera	Problema	Corrección
<code>SQLite.openDatabase('db')</code> o <code>db.transaction(tx => tx.executeSql(...))</code>	API antigua de expo-sqlite, retirada	<code>openDatabaseSync</code> o <code>openDatabaseAsync</code>
<code>import { drizzle } from 'drizzle-orm/better-sqlite3'</code>	Driver de Node, no funciona en el celular	<code>drizzle-orm/expo-sqlite</code>
<code>drizzle.config.ts</code> sin <code>driver: 'expo'</code>	No genera <code>migrations.js</code>	Agregar <code>driver: 'expo'</code>
Falta el plugin <code>inline-import</code> o la extensión <code>sql</code> en Metro	Error al importar migraciones	Configurar Babel y Metro (Paso 2)
<code>npx drizzle-kit push</code>	Pensado para bases de servidor	<code>npx drizzle-kit generate + useMigrations</code>

La IA genera	Problema	Corrección
Abrir la base sin <code>enableChangeListener</code>	<code>useLiveQuery</code> no se actualiza	<code>openDatabaseSync(nombre, { enableChangeListener: true })</code>
Dinero en <code>real</code>	Errores de redondeo	Enteros en la unidad más pequeña
Llaves foráneas sin <code>PRAGMA foreign_keys = ON</code>	SQLite no las hace cumplir	Ejecutar el PRAGMA al abrir la conexión
<code>FileSystem.copyAsync</code> mezclado con <code>new File()</code>	Mezcla de API anterior y nueva	Usar una sola API según el SDK

Ejcm2lo dc AI-LOG.md

```
# Registro de Auditoría de IA (AI-LOG)

**Estudiante(s):** Nombre Apellido, Nombre Apellido
**Semana:** 7
**Proyecto:** GeoCam persistente - Taller Integrador 2

## 1. Prompts Utilizados
- "Crea con Drizzle ORM y expo-sqlite una tabla albums relacionada con
  photos,
  de forma que al borrar un álbum las fotos no se eliminen."

## 2. Código Generado vs. Código Modificado
- **¿Qué generó la IA?:** El esquema con references() correcto, pero con
  onDelete: 'cascade', que borraba las fotos junto con el álbum.
- **¿Qué modifiqué/corregí?:** Cambié a onDelete: 'set null', dejé albumId
  opcional y agregué PRAGMA foreign_keys = ON en el cliente, porque sin él
  SQLite ignoraba la regla.

## 3. Alucinaciones o Errores Detectados
- La IA configuró el cliente con `drizzle-orm/better-sqlite3`, que es un
  driver para Node. Lo reemplacé por `drizzle-orm/expo-sqlite` con
  openDatabaseSync según el quickstart oficial de Drizzle para Expo.
```

Documentación de referencia

- [Expo SQLite](#)
- [Drizzle ORM: quickstart con Expo SQLite](#)
- [Drizzle ORM: consultas select](#)
- [Expo FileSystem](#)

Para la próxima semana: crea un proyecto en Supabase y lee el quickstart de Expo con React Native. Piensa en esta pregunta: si cambias de celular, ¿dónde quedan tus fotos?

